Chapter Three

Control Statements

Introduction

- C++ provides different forms of statements for different purposes
  - Declaration statements
  - Assignment-like statements
  - etc
- The order in which statements are executed is called **flow control**
- Branching/selection/decision making statements
  - specify alternate paths of execution, depending on the outcome of a logical condition.
  - E.g. if, if..else, switch…case
- Loop statements
  - specify computations, which need to be repeated until a certain logical condition is satisfied.
  - E.g. for ,while, do…while

Branching Mechanisms

- if-else statements
  - Choice of two alternate statements based on condition expression.
  - Example:

```
if (hrs > 40)
        grossPay = rate*40 + 1.5*rate*(hrs-40);
else
        grossPay = rate*hrs;
```

if-else Statement Syntax

- Formal syntax:
  if (<boolean_expression>)
  <yes_statement>
  else
  <no_statement>
- Note each alternative is only ONE statement!
- To have multiple statements execute in either branch → use compound statement

Compound/Block Statement

- Only "get" one statement per branch
- Must use compound statement { }
  for multiples
  - Also called a "block" statement
- Each block should have block statement
  - Even if just one statement
  - Enhances readability

Compound Statement in Action

- Note indenting in this example:

```
if (myScore > yourScore)
{
        cout << "I win!\n";
        wager = wager + 100;
}
else
{
        cout << "I wish these were golf scores.\n";
        wager = 0;
}
```

program to check if a number is positive or negative

```
// #include <iostream>
using namespace std;
int main() {
    int n;
    cout << "Enter a number: ";
    cin >> n;
    if (n > 0) {
        cout << "Positive";
    } else {
        cout << "Negative or Zero";
    }
    return 0;
}
```

```
//using function
#include <iostream.h>
void checkNumber(int n)
{
 if (n > 0) {
 cout << "Positive";
 }
else {
cout << "Negative or Zero";
  }
}
int main() {
 int n;
 cout << "Enter a number: ";
 cin >> n;
checkNumber(n);
 return 0;}
```

C++ rogram to check if a number is even or odd

- #include <iostream>

```
using namespace std;
int main() {
int n;
   cout << "Enter a number: ";
  cin >> n;
if (n % 2 == 0)
{
 cout << "Even";
 }
else
{
   cout << "Odd";
}
  return 0;
}
```

- #include <iostream.h>

```
void checkEvenOdd(int x)
int main() {
 int n;
 cout << "Enter a number: ";
 cin >> n;
checkEvenOdd(n);
    return 0;
}
void checkEvenOdd(int n) {
if (n % 2 == 0) {
cout << "Even";
  } else {
cout << "Odd";
 }
}
```

Common Pitfalls

- Operator "=" vs. operator "=="
- One means "assignment" (=)
- Two means "equality" (==)
  - VERY different in C++!
  - Example:
    if (x = 12) ←Note operator used!
        Do_Something
    else
        Do_Something_Else

The Optional else

- else clause is optional
  - If, in the false branch (else), you want "nothing" to happen, leave it out
  - Example:
    ```
    if (sales >= minimum)
        salary = salary + bonus;
    cout << "Salary = %" << salary;
    ```
  - Note: nothing to do for false condition, so there is no else clause!
  - Execution continues with cout statement

Nested Statements

- if-else statements contain smaller statements
  - Compound or simple statements (we've seen)
  - Can also contain any statement at all, including another if-else stmt!
  - Example:

```
if (speed > 55)
    if (speed > 80)
        cout << "You're really speeding!";
    else
        cout << "You're speeding.";
```

    - Note proper indenting!

Program to Find the largest among three numbers

```
#include <iostream.h>
int main() {
 int a, b, c;
 cout << "Enter three numbers: ";
 cin >> a >> b >> c;
 if (a > b) {
 if (a > c) {
 cout << "Largest is " << a;
 } else {
  cout << "Largest is " << c;
 }  }
else {
  if (b > c) {
      cout << "Largest is " << b;
  } else
{         cout << "Largest is " << c;
  }    }
 return 0;}
```

```
#include <iostream>
void findLargest(int a, int b, int c) { if (a > b) {
        if (a > c) {
            cout << "Largest is " << a; }
else { cout << "Largest is " << c;}
}
else {   if (b > c) {
 cout << "Largest is " << b;}
else {
  cout << "Largest is " << c; }
}}
int main() {
    int x, y, z;
    cout << "Enter three numbers: ";
    cin >> x >> y >> z;
    findLargest(x, y, z);
    return 0;}
```

Multiway if-else: Display

- Not new, just different indenting
- Avoids "excessive" indenting
  - Syntax:

Multiway if-else Statement

SYNTAX

```
if (Boolean_Expression_1)
    Statement_1
else if (Boolean_Expression_2)
    Statement_2
          .
          .
          .
else if (Boolean_Expression_n)
    Statement_n
else
    Statement_For_All_Other_Possibilities
```

Multiway if-else Example: Display

EXAMPLE

```
if ((temperature < -10) && (day == SUNDAY))
    cout << "Stay home.";
else if (temperature < -10) //and day != SUNDAY
    cout << "Stay home, but call work.";
else if (temperature <= 0) //and temperature >= -10
    cout << "Dress warm.";
else //temperature > 0
    cout << "Work hard and play hard.";
```

The Boolean expressions are checked in order until the first `true` Boolean expression is encountered, and then the corresponding statement is executed. If none of the Boolean expressions is *true*, then the *Statement_For_All_Other_Possibilities* is executed.

- // program to compute grade without function

```
#include <iostream>
using namespace std;
int main() {
  int score;
 cout << "Enter score: ";
 cin >> score;
 if (score >= 90) {
 cout << "Grade = A"; }
else if (score >= 80) {
     cout << "Grade = B"; }
else if (score >= 70) {
cout << "Grade = C";}
 else if (score >= 60) {
 cout << "Grade = D";}
else {
cout << "Grade = F"; }
  return 0;}
```

- #include <iostream.h>

```
char computeGrade(int score) {
  if (score >= 90) {
        return 'A';}
else if (score >= 80) {
        return 'B';}
else if (score >= 70) {
         return 'C';}
else if (score >= 60) {
       return 'D'; }
else {    return 'F'; }
}// close function
int main() {
  int mark;
 cout << "Enter score: ";
  cin >> mark;
 char grade = computeGrade(mark);
 cout << "Grade = " << grade;
 return 0;}
```

The switch Statement

- A new statement for controlling multiple branches
- Uses controlling expression which returns bool data type (true or false)
- Syntax:
  - next slide

switch Statement Syntax

switch Statement

SYNTAX

```
switch (Controlling_Expression)
{
    case Constant_1:
        Statement_Sequence_1
        break;
    case Constant_2:
        Statement_Sequence_2
        break;
                    .
                    .
                    .
    case Constant_n:
        Statement_Sequence_n
        break;
    default:
        Default_Statement_Sequence
}
```

You need not place a `break` *statement in each case. If you omit a* `break`*, that case continues until a* `break` *(or the end of the* `switch` *statement) is reached.*

The switch Statement in Action

EXAMPLE

```
int vehicleClass;
double toll;
cout << "Enter vehicle class: ";
cin >> vehicleClass;

switch (vehicleClass)
{
    case 1:
        cout << "Passenger car.";
        toll = 0.50;
        break;
    case 2:
        cout << "Bus.";
        toll = 1.50;
        break;
    case 3:
        cout << "Truck.";
        toll = 2.00;
        break;
    default:
        cout << "Unknown vehicle class!";
  }
```

If you forget this **break**, *then passenger cars will pay $1.50.*

The switch: multiple case labels

- Execution "falls thru" until break
  - switch provides a "point of entry"
  - Example:

```
case "A":
case "a":
    cout << "Excellent: you got an "A"!\n";
    break;
case "B":
case "b":
    cout << "Good: you got a "B"!\n";
    break;
```

  - Note multiple labels provide same "entry"

Menu design using switch case

- #include <iostream>
- Using namespace std;

```
int main() {
 int choice;
 cout << "1. Add\n  2. Subtract\n3. Multiply\n4. Divide\n";
  cout << "Enter choice: ";
  cin >> choice;
    switch (choice) {
  case 1:
  cout << "You selected Add";
     break;
   case 2:
   cout << "You selected Subtract";
break;
case 3:
cout << "You selected Multiply";
break;
case 4:
cout << "You selected Divide";
  break;
default:
 cout << "Invalid choice";
}  //close switch case
   return 0;
}
```

Exercise

- Write a simple Calculator program using switch.
- Write a program for grade message. Assume tha t case A is "Excellent!“, B is Very Good, C “Good“, D is "Pass“, F is "Fail"; other wise "Unknown grade";

Answers

- #include <iostream.h>

```
int main() {
 char op;
 int a = 10, b = 5;
    cout << "Enter operator (+, -, *, /): ";
 cin >> op;
    switch (op) {
      case '+': cout << "Sum = " << a + b;
      break;
 case '-': cout << "Difference = " << a - b;
     break;
case '*':   cout << "Product = " << a * b;
          break;
 case '/': cout << "Quotient = " << a / b;
break;
default: cout << "Invalid operator";
 }
}
```

- Q2:

```
#include<iostrream.h>
Int main()
{
char grade = 'C';
switch (grade) {
case 'A': cout << "Excellent!";
break;
case 'B': cout << "Very Good";
break;
case 'C': cout << "Good"; break;
case 'D': cout << "Pass"; break;
case 'F': cout << "Fail"; break;
 default: cout << "Unknown grade";
}}
```

switch Pitfalls/Tip

- Forgetting the break;
  - No compiler error
  - Execution simply "falls thru" other cases until break;
- Biggest use: MENUs
  - Provides clearer "big-picture" view
  - Shows menu structure effectively
  - Each branch is one menu choice

switch Menu Example

- Switch statement "perfect" for menus:

```
switch (response)
{
        case "1":
                // Execute menu option 1
                break;
        case "2":
                // Execute menu option 2
                break;
        case 3":
                // Execute menu option 3
                break;
        default:
                cout << "Please enter valid response.";
}
```

Conditional Operator

- Also called "ternary operator"
  - Allows embedded conditional in expression
  - Essentially "shorthand if-else" operator
  - Example:
    ```
    if (n1 > n2)
        max = n1;
    else
        max = n2;
    ```
  - Can be written:
    ```
    max = (n1 > n2) ? n1 : n2;
    ```
    - "?" and ":" form this "ternary" operator

Ternay example

```
#include <iostream>
using namespace std;
int main() {
  int num = 7;
   cout << (num % 2 == 0 ? "Even" : "Odd");
int a = 10, b = 5;
int result = (a > b) ? (a + b) : (a - b);
int  result_2 = (a > b) ? (++a) : (--b);
int x = 4, y = 2, z;
int r = (x % 2 == 0) ? (x * y + 10) : (x / y - 5);
z = (x < y) ? (x = x + 5) : (y = y - 5);
(x > 10) ? cout << "Greater" : cout << "Not greater";
}
```

Loops

- 3 Types of loops in C++
  - while
    - Most flexible
    - No "restrictions"
  - do -while
    - Least flexible
    - Always executes loop body at least once
  - for
    - Natural "counting" loop

while Loops Syntax

Syntax for `while` and `do-while` Statements

A `while` STATEMENT WITH A SINGLE STATEMENT BODY

```
while (Boolean_Expression)
    Statement
```

A `while` STATEMENT WITH A MULTISTATEMENT BODY

```
while (Boolean_Expression)
{
    Statement_1
    Statement_2
      .
      .
      .
    Statement_Last
}
```

while Loop Example

- Consider:

```
count = 0;                  // Initialization
while (count < 3)           // Loop Condition
{
    cout << "Hi ";          // Loop Body
    count++;        // Update expression
}
```

  - Loop body executes how many times?

- //Sum of First N Numbers

```
#include <iostream>
using namespace std;
int main() {
 int n, sum = 0, i = 1;
 cout << "Enter a number: ";
  cin >> n;
  while (i <= n) {
 sum = sum + i;
  i++;
 }
    cout << "Sum = " << sum;
 return 0;
```

- // User Input (Until 0 is Entered)
- #include <iostream>

```
using namespace std;
int main() {
    int num;
  cout << "Enter numbers (0 to stop): ";
   cin >> num;
while (num != 0) {
   cout << "You entered: " << num << endl;
    cin >> num; // take next input
    }
  cout << "Loop ended.";
   return 0;
}
```

do-while Loop Syntax

A do-while STATEMENT WITH A SINGLE-STATEMENT BODY

```
do
    Statement
while (Boolean_Expression);
```

A do-while STATEMENT WITH A MULTISTATEMENT BODY

```
do
{
    Statement_1
    Statement_2
      .
      .
      .
    Statement_Last
} while (Boolean_Expression);
```

Do not forget the final semicolon.

do-while Loop Example

- ```
  count = 0;          // Initialization
  do
  {
      cout << "Hi ";      // Loop Body
      count++;            // Update expression
  } while (count < 3);    // Loop Condition
  ```
  - Loop body executes how many times?
  - do-while loops always execute body at least once!
  ```

- #include <iostream>

```
using namespace std;
int main() {
   int i = 10;
  do {
    cout << "This runs once even though i > 5" << endl;
 } while (i <= 5);
    return 0;
}
```

- #include <iostream>

```
using namespace std;
int main() {
   int num, sum = 0;
 do {
  cout << "Enter a number (0 to stop): ";
 cin >> num;
 sum += num;
 } while (num != 0);
 cout << "Sum = " << sum;
 return 0;
}
```

while vs. do-while

- Very similar, but…
  - One important difference
    - Issue is "WHEN" boolean expression is checked
    - while: checks BEFORE body is executed
    - do-while: checked AFTER body is executed
- After this difference, they're essentially identical!
- while is more common, due to it's ultimate "flexibility"

for Loop Syntax

```
for (Init_Action; Bool_Exp; Update_Action)
    Body_Statement
```

- Like if-else, Body_Statement can be a block statement
  - Much more typical

for Loop Example

- for (count=0;count<3;count++)
  {
  cout << "Hi "; // Loop Body
  }
- How many times does loop body execute?
- Initialization, loop condition and update all "built into" the for-loop structure!
- A natural "counting" loop

Converting Between For and While Loops

```
for (int i = 1; i < 1024; i *= 2) {
  cout << i << endl;
}
```

```
int i = 1;
while (i < 1024) {
   cout << i << endl;
   i *= 2;
}
```

Loop Issues

- Loop's condition expression can be ANY boolean expression
- Examples:

```
while (count<3 && done!=0)
{
    // Do something
}
for (index=0;index<10 && entry!=-99)
{
    // Do something
}
```

Loop Pitfalls: Misplaced ;

- Watch the misplaced ; (semicolon)
  - Example:

```
while (response != 0) ;←
{
    cout << "Enter val: ";
    cin >> response;
}
```

  - Notice the ";" after the while condition!
- Result here: INFINITE LOOP!

Loop Pitfalls: Infinite Loops

- Loop condition must evaluate to false at some iteration through loop
  - If not → infinite loop.
  - Example:

```
while (1)
{
    cout << "Hello ";
}
```

  - A perfectly legal C++ loop → always infinite!
- Infinite loops can be desirable
  - e.g., "Embedded Systems"

The break and continue Statements

- Flow of Control
  - Recall how loops provide "graceful" and clear flow of control in and out
  - In RARE instances, can alter natural flow
- break;
  - Forces loop to exit immediately.
- continue;
  - Skips rest of loop body
- These statements violate natural flow
  - Only used when absolutely necessary!

Example

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) break;
}
```

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;
    cout << i << " ";
}
```

Nested Loops

- Recall: ANY valid C++ statements can be inside body of loop
- This includes additional loop statements!
  - Called "nested loops"
- Requires careful indenting:

```
for (outer=0; outer<5; outer++)
    for (inner=7; inner>2; inner--)
        cout << outer << inner;
```

  - Notice no { } since each body is one statement
  - Good style dictates we use { } anyway

- #include <iostream>

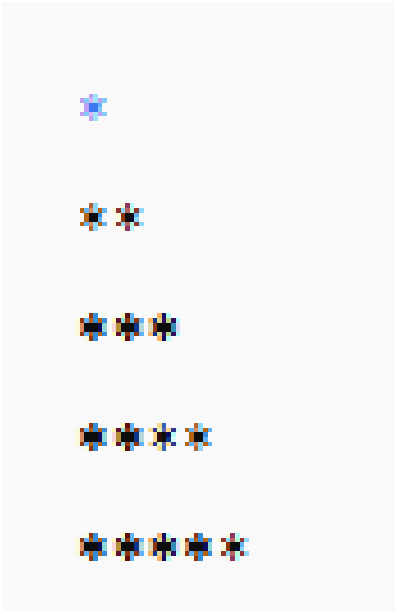
```
using namespace std;
int main() {
  for (int i = 1; i <= 3; i++) {
// Outer loop
 cout << "Outer loop i = " << i
<< endl;
 for (int j = 1; j <= 2; j++) {
// Inner loop
cout << "   Inner loop j = " << j
<< endl;
 }
 cout << endl; // just for better
formatting
 }
 return 0;}
```

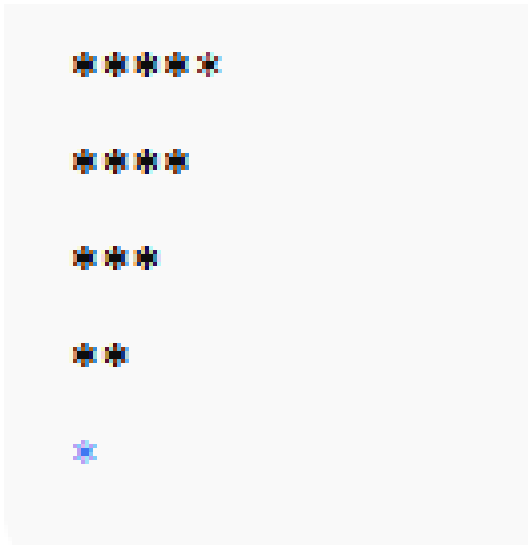
- #include <iostream>

```
using namespace std;
int main() {
   for (int i = 1; i <= 3; i++) {
// Outer for loop
  cout << "Outer loop i = " << i <<
endl;
       int j = 1;
       while (j <= 2) {
cout << "   Inner while j = " << j <<
j++;
 }
       cout << endl;
 }
  return 0;
}
```

Exercise

- Write a C++ program to compute average of natural language from 100 500.
- Write a program to display even natural number from 1000 to 600 in descending order.
- Write a program to accept 50 students' name, age, mark from a user?
- Write separate programs to display the following outputs.





```
• #include <iostream>
int main()
{  int start = 100, end = 500;
   int sum = 0;
 int count = 0;
for (int i = start; i <= end; i++)
{
    sum += i;
 count++;
  }
    double average = (double)sum /
count;


   cout << "The average of natural
numbers from 100 to 500 is: " <<
average << endl;
return 0;
}
```

Even number between 1000 and 600

- **Method 1:**

```
#include <iostream>
using namespace std;
int main() {
for(int i = 1000; i >= 600; i -= 2)
{
  cout << i << " ";
   }
  return 0;
}
```

Method 2:

```
#include <iostream>
using namespace std;
int main() {
    for (int i = 1000; i >= 600; i--)
{
        if (i % 2 == 0)
{
cout << i << " ";
     }
    }
    return 0;
}
```

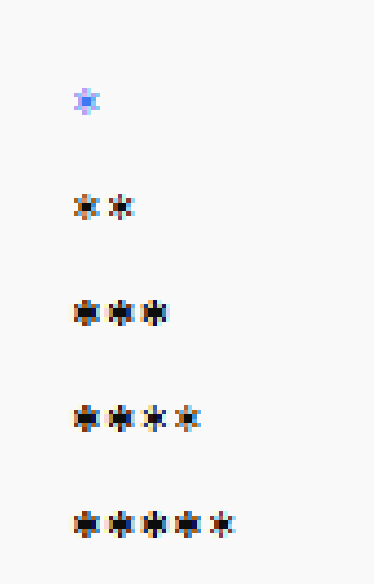
program to accept 50 students' name, age, mark from a user?

```
#include <iostream>
using namespace std;
int main() {
   string name;
  int age;
  float mark;
 cout << "Enter name, age and mark for 50 students:\n";
 for(int i = 1; i <= 50; i++) {
  cout << "\nStudent " << i << endl;
   cout << "Enter name: ";
   cin >> name;
  cout << "Enter age: ";
   cin >> age;
 cout << "Enter mark: ";
cin >> mark;
// Immediately display
cout << "\nYou entered----\n";
cout << "Name : " << name << endl;
 cout << "Age  : " << age << endl;
  cout << "Mark : " << mark << endl;
   }
 return 0;
}
```

Display *

- ```
  #include <iostream>
  using namespace std;

  int main() {
      int n = 5;
      for(int i = 1; i <= n; i++) {
          for(int j = 1; j <= i; j++) {
              cout << "*";
          }
          cout << endl
      }
      return 0;
  }
  ```



- ```
  #include <iostream>
  using namespace std;
  int main() {
      for(int i = 5; i >= 1; i--) {       //
  rows
    for(int j = 1; j <= i; j++) {    //
  stars in each row
       cout << "*";
        }
       cout << endl;
    }
     return 0;
  }
  ```